Polytechnic of Porto - School of Engineering (ISEP)

Telecommunications and Informatics Engineering
Embedded Systems (SISEM)

Final Project

GPS Module Integration with Multi-Peripheral System

XMC4200 , AHT10 , CAN , LCD , GPS, EEPROM

Members: Diogo Nogueira - 1241692
Vasco Magolo - 1231562

Group: 3

Unit: LETI-SISEM

Date: June 2026

Contents

1. Topic Overview

The assigned work is **Trabalho 5 - Leitura e transmissão de dados GPS com GY-GPS6MV2**. The objective is to develop a system capable of reading NMEA 0183 sentences from a u-blox NEO-6M GPS module over UART, parsing the relevant fields from `$GPGGA` sentences (latitude, longitude, UTC time, fix quality, satellite count), and forwarding that information in real time - to a UART terminal via Docklight and over the CAN bus.

This work is built on top of the multi-peripheral platform developed during Tasks A and B, preserving all earlier functionality. The GPS integration adds a seventh feature module to the system:

Feature	Description	Task
LED + Button	Blink rate configurable via button press or CLI	A
Potentiometer	8-bit ADC reading drives PWM duty cycle	A
AHT10	I2C temperature and humidity sensor	B
CAN Bus	Periodic sensor frames; receive and decode frames from others	B
LCD 16x2	I2C display with six selectable modes	B
EEPROM	AT24C32E persists settings and LCD text across power cycles	B
GPS	NMEA parsing via UART - lat, lon, UTC, fix status, satellites	Final

Table 1: Implemented features per development task

All feature modules are independently gated by flags in `config.h`:

```c
#define FEATURE_AHT10   // I2C temperature and humidity sensor
#define FEATURE_CAN     // CAN bus transmit and receive
#define FEATURE_LCD     // 16x2 LCD display over I2C
#define FEATURE_GPS     // GPS module via UART
#define FEATURE_EEPROM  // AT24C32E settings persistence over I2C
```

Commenting out any flag removes all related code - includes, periodic tasks, ISR handlers, and CLI menu options - at compile time, without touching any other file.

2. Code Architecture

2.1. File Structure

The firmware is organised into self-contained modules under `lib/`. The entry point and event loop live in `main.c`; shared mutable state lives in `app_state`; all peripheral drivers are under `lib/`.

```
(root)/
├── config.h          - feature flags (#define FEATURE_*)
├── app_state.{c,h}   - shared volatile state (potentiometer, led_blinking,
timer_interval)
├── main.c            - DAVE_Init, system_tick ISR, main event loop
└── lib/
    ├── gps.{c,h}     - NMEA character parser, gps_data_t struct, gps_feed()
    ├── can.{c,h}     - CAN send/receive, sensor and GPS frame encoding/decoding
    ├── cli.{c,h}     - UART CLI state machine, EEPROM save/load
    ├── uart.{c,h}    - uart_printf() convenience wrapper over DAVE UART APP
    ├── aht10.{c,h}   - I2C AHT10 driver (trigger, read, parse humidity/
    temperature)
    ├── lcd.{c,h}     - HD44780 LCD driver via PCF8574 I2C expander
    ├── i2c.{c,h}     - Blocking I2C transmit/receive wrapper
    └── eeprom.{c,h}  - AT24C32E EEPROM read/write over I2C
```

2.2. Tick-Driven Event Dispatch

The 10 ms hardware timer ISR (`system_tick`) is the sole source of timing for the entire system. The ISR never blocks - it sets `volatile bool` flags and performs the minimum LED toggle calculation. The main loop polls those flags and dispatches all work:

- `adc_tick` - set every tick → ADC read, PWM update, optional LCD refresh
- `btn_event` - set on button rising edge → toggle `led_blinking`
- `can_sensor_tick` - set every 50 ticks (500 ms) → read AHT10, send CAN frame

GPS and CLI bytes are polled directly from hardware FIFOs as the FIFO itself buffers incoming bytes between loop iterations, so no additional flag is needed.

2.3. Main Loop Flowchart

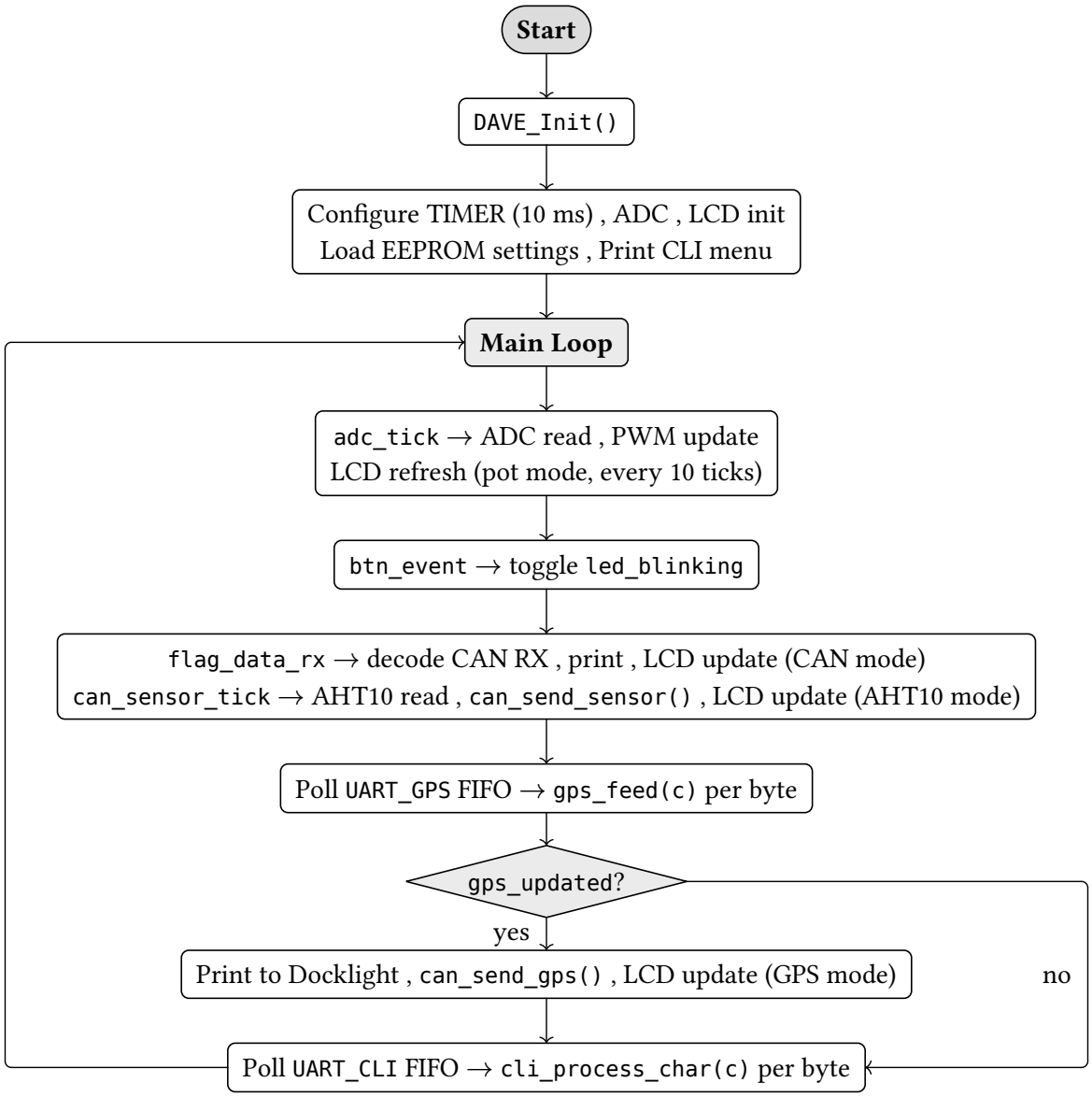

Figure 1: Main loop event dispatch - all features enabled

3. Peripheral Modules

3.1. LED and Button

The LED is driven entirely from the tick ISR. When `led_blinking` is false the output is held high. When true, the ISR toggles the pin every `led_tick_interval` ticks. Button press detection uses a single `static bool btn_prev` to detect the rising edge. A press sets `btn_event`, which the main loop clears and uses to invert `led_blinking`.

The blink period is stored in `timer_interval` (ms). The half-period in ticks is `led_tick_interval = (timer_interval / 2) / TICK_MS`. Default: 2 s (100 ticks × 2 half-periods × 10 ms).

3.2. Potentiometer (ADC + PWM)

An 8-bit ADC sample is taken every tick. The value scales linearly to the DAVE PWM APP's fixed-point duty cycle format (0 = 0.00 %, 10000 = 100.00 %):

```c
PWM_SetDutyCycle(&PWM_POTENTIOMETER,
    (uint32_t)potentiometer_value * 10000 / 255);
```

3.3. AHT10 Temperature and Humidity Sensor

The AHT10 communicates over I2C at address 0x38. The driver sends a 3-byte measurement command (`0xAC`, `0x33`, `0x00`) then waits approximately 80 ms for conversion before reading 6 bytes. The 20-bit humidity value occupies bytes 1-3 (high nibble) and the 20-bit temperature value occupies bytes 3-5 (low nibble):

```c
void aht10_parse_temperature(volatile float *out, uint8_t data[]) {
    uint32_t t_part1 = ((uint32_t) (data[3] & 0x0F)) << 16;
    uint32_t t_part2 = (uint32_t) data[4] << 8;
    uint32_t t_part3 = (uint32_t) data[5];

    uint32_t t_raw = t_part1 | t_part2 | t_part3;

    *out = ((float) t_raw * 200.0f) / (float) (1 << 20) - 50.0f;
}

void aht10_parse_humidity(volatile float *out, uint8_t data[]) {
    uint32_t h_part1 = (uint32_t) data[1] << 12;
    uint32_t h_part2 = (uint32_t) data[2] << 4;
    uint32_t h_part3 = (uint32_t) data[3] >> 4;

    uint32_t h_raw = h_part1 | h_part2 | h_part3;

    *out = ((float) h_raw * 100.0f) / (float) (1 << 20);
}
```

The `volatile float *out` parameter matches the `volatile float` globals `temperature` and `humidity` that are written in the main loop and read asynchronously.

3.4. CAN Bus

The CAN bus operates at 500 kbps. This node transmits two 8-byte frame types on our group's assigned CAN ID `0x4C0`:

Bytes	Signal	Encoding
0-3	Temperature	`uint32` = (degrees + 55) × 10 *(factor 0.1, offset –55)*
4-7	Humidity	`uint32` = humidity × 10 *(factor 0.1, offset 0)*
0-3	Latitude	`int32` = degrees × 10^6 *(factor* 10^{-6}*, signed)*
4-7	Longitude	`int32` = degrees × 10^6 *(factor* 10^{-6}*, signed)*

Table 2: CAN frames transmitted by Group 3

The sensor frame is sent every 500 ms. The GPS frame is sent on each valid fix update (1 Hz, only when `gps_data.fix_valid` is true). Both frames share the same CAN ID and are distinguished by context - the sensor frame is periodic, the GPS frame is event-driven.

The temperature encoding uses a +55 offset so that the full AHT10 sensor range (-40 °C to +85 °C) fits in an unsigned 32-bit integer. GPS coordinates use a signed `int32_t` × 10^6 representation, giving approximately 0.11 m resolution.

Receive is interrupt-driven: `can_interrupt()` fires when message object 0 receives a frame, copies the payload into `data_rx`, and sets `flag_data_rx`. The main loop dispatches to `cli_process_can_rx`, which applies the CLI-configured software filter before printing the decoded temperature and humidity to Docklight.

3.5. LCD Display

The 16x2 LCD uses the HD44780 controller behind a PCF8574 I2C I/O expander at address 0x27, sharing the I2C bus with the AHT10 and EEPROM. The driver transmits 4-bit nibbles, toggling the `EN` line between writes. Six display modes are selectable via CLI command `6`:

Mode	Cmd	Content
`LCD_MODE_OFF`	`6→0`	Display cleared
`LCD_MODE_CAN_RX_AHT10`	`6→1`	Last CAN RX frame: CAN ID on row 1, Temp/Hum on row 2
`LCD_MODE_POT`	`6→2`	Potentiometer ADC value, refreshed every 100 ms
`LCD_MODE_GPS`	`6→3`	Lat/Lon when fix valid; "No GPS fix" + satellite count otherwise
`LCD_MODE_LOCAL_AHT10`	`6→4`	Local AHT10 temperature and humidity
`LCD_MODE_EEPROM`	`6→5`	Two custom text rows stored in EEPROM (written via CLI `7`/`8`)

Table 3: LCD display modes

Each mode in the main loop tracks previously displayed values in `static` variables and skips the I2C write when the content has not changed, preventing unnecessary LCD flicker.

3.6. GPS Module

The GY-GPS6MV2 board (u-blox NEO-6M chip) connects to the XMC4200 MikroBUS socket via a dedicated UART (`UART_GPS`) at 9600 baud, 8N1. The module emits six NMEA 0183 sentence types every second; only `$GPGGA` is parsed and all others are silently discarded.

Parameter	Value
Chip	u-blox NEO-6M
Interface	UART 9600 8N1
Protocol	NMEA 0183
Update rate	1 Hz
Supply voltage	3.3 V - 5 V
Sentences parsed	`$GPGGA` only
TTFF (cold start)	1 - 5 min (outdoor, clear sky)

Table 4: GY-GPS6MV2 / u-blox NEO-6M module characteristics

The `$GPGGA` sentence format and the fields extracted:

```
1  $GPGGA,HHMMSS.ss,LLLL.LL,a,YYYYY.YY,b,q,nn,...
```

Field	Index	Description
`HHMMSS`	`fields[1]`	UTC time - hours, minutes, seconds
`LLLL.LL`	`fields[2]`	Latitude in DDDMM.MMMM format
`a`	`fields[3]`	N/S hemisphere indicator
`YYYYY.YY`	`fields[4]`	Longitude in DDDMM.MMMM format
`b`	`fields[5]`	E/W hemisphere indicator
`q`	`fields[6]`	Fix quality (0 = no fix, $\geq$ 1 = valid fix)
`nn`	`fields[7]`	Number of satellites in use

Table 5: Fields extracted from the `$GPGGA` sentence

When `fix_valid` is true the parsed coordinates are printed to Docklight and forwarded on CAN. No coordinates are stored or transmitted during a fix loss - only the satellite count and the no-fix indication are updated. The LCD GPS mode redraws only when latitude, longitude, or satellite count changes.

3.7. EEPROM Settings Persistence

The AT24C32E is a 4 096-byte I2C EEPROM at address 0xA0. A 9-byte header at address `0x0000` persists all user-configurable state:

Offset	Size	Field	Description
`0x00`	1 B	`magic`	Always `0xAB` - marks the header as valid
`0x01`	4 B	`timer_interval`	LED blink period in ms (100 - 10 000)
`0x05`	2 B	`can_filter_id`	Active CAN RX filter ID (11-bit)
`0x07`	1 B	`can_filter_active`	1 if CAN filter is enabled, 0 otherwise
`0x08`	1 B	`lcd_mode`	Last selected LCD mode (0 - 5)

Table 6: EEPROM header layout (9 bytes at 0x0000)

Two 16-byte slots follow the header: `0x0009` for LCD row 1 and `0x0019` for LCD row 2. These are written via CLI commands `7` and `8`.

On boot, `cli_load_settings()` reads the header and validates the magic byte. If it matches `0xAB`, all settings are restored into `app_state` globals. If not (first power-on or data corruption), the firmware starts with compile-time defaults. Settings are automatically re-saved after every CLI command that changes them.

4. Relevant Code

4.1. System Tick ISR

The 10 ms ISR sets all event flags and handles LED toggling without blocking:

```c
void system_tick(void) {
    tick_count++;

    adc_tick = true;

    static bool btn_prev = false;
    bool btn_now = (DIGITAL_IO_GetInput(&DIGITAL_IO_BTN) == 1);
    if (btn_now && !btn_prev) {
        btn_event = true;
    }
    btn_prev = btn_now;

#   if defined(FEATURE_CAN)
        if (tick_count % CAN_SENSOR_TICKS == 0) {
            can_sensor_tick = true;
        }
#   endif

    if (!led_blinking) {
        DIGITAL_IO_SetOutputHigh(&DIGITAL_IO_LED);
    } else if (tick_count % led_tick_interval == 0) {
        DIGITAL_IO_ToggleOutput(&DIGITAL_IO_LED);
    }
}
```

`CAN_SENSOR_TICKS` is `500 / TICK_MS = 50`. The LED half-period (`led_tick_interval`) is computed from the CLI-set `timer_interval` as `(timer_interval / 2) / TICK_MS`.

4.2. GPS Character Parser - `gps_feed()`

Called for every byte polled from the `UART_GPS` FIFO. A `$` resets the buffer and `\n` triggers sentence identification. Only `$GPGGA` sentences are forwarded to `parse_gpgga()`, while all others are discarded silently:

```c
void gps_feed(char c) {
    if (c == '$') {
        buf_len = 0;
        buf[buf_len++] = c;
        return;
    }

    if (buf_len == 0) return;
    if (buf_len < GPS_BUF_SIZE - 1) buf[buf_len++] = c;

    if (c == '\n') {
        buf[buf_len] = '\0';
        while (buf_len > 0 && (buf[buf_len-1] == '\r' || buf[buf_len-1] == '\n')) {
            buf[--buf_len] = '\0';
        }
        if (strncmp(buf, "$GPGGA,", 7) == 0) {
            parse_gpgga(buf)
        }
        buf_len = 0;
    }
}
```

Flowchart of the character state machine:

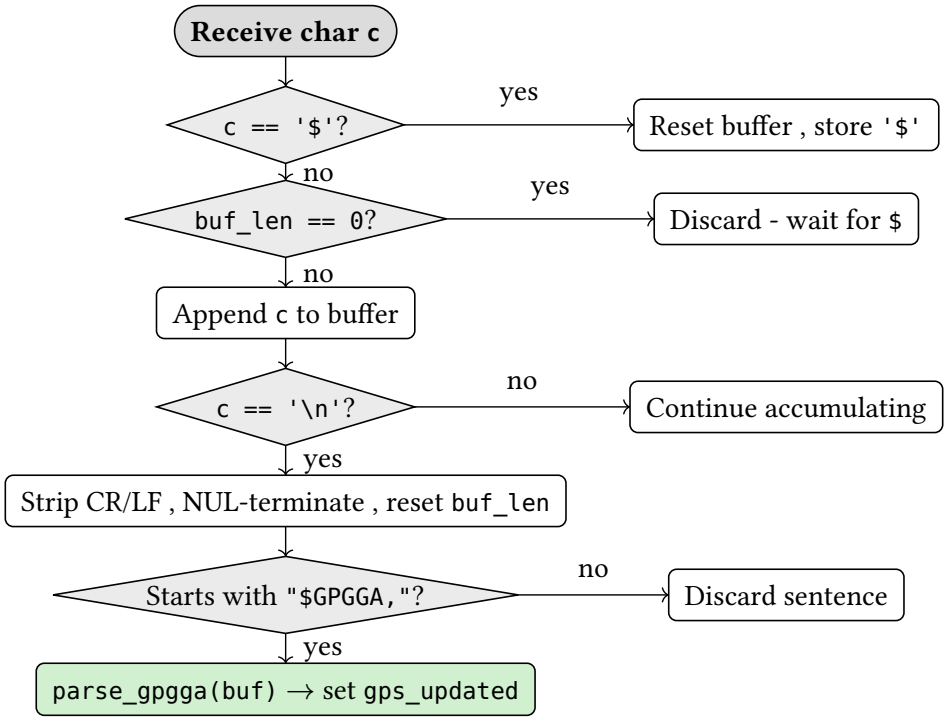

Figure 2: `gps_feed()` character state machine

4.3. GPGGA Coordinate Parser - `parse_coord()`

The `parse_gpgga` function tokenizes the sentence by replacing commas with `\0`. Coordinate fields arrive in DDDMM.MMMM format and the function `parse_coord` converts them to decimal degrees and applies the hemisphere sign:

```c
static float parse_coord(const char *val, char dir) {
    if (!val || val[0] == '\0') return 0.0f;
    float raw    = (float)atof(val);
    int   deg    = (int)(raw / 100.0f);
    float min    = raw - (float)(deg * 100);
    float result = (float)deg + min / 60.0f;
    if (dir == 'S' || dir == 'W') result = -result;
    return result;
}
```

The fix quality and satellite count are always written to `gps_data`. Coordinates are only updated when `fix_valid` is true, preventing stale data from being published on CAN after a fix is lost.

4.4. CAN Frame Encoding

Both sensor and GPS frames are packed little-endian into 8-byte payloads. The sensor frame uses an unsigned encoding with a +55 °C offset; the GPS frame uses a signed `int32_t` $\times 10^6$:

```c
void can_send_sensor(const CAN_NODE_t *can_node, uint16_t can_id,
                     float temperature, float humidity) {
    uint32_t temp_enc = (uint32_t)((temperature + 55.0f) * 10.0f);
    uint32_t hum_enc  = (uint32_t)(humidity * 10.0f);
    uint8_t payload[8];
    payload[0] = (uint8_t)(temp_enc);
    payload[1] = (uint8_t)(temp_enc >> 8);
    payload[2] = (uint8_t)(temp_enc >> 16);
    payload[3] = (uint8_t)(temp_enc >> 24);
    payload[4] = (uint8_t)(hum_enc);
    payload[5] = (uint8_t)(hum_enc >> 8);
    payload[6] = (uint8_t)(hum_enc >> 16);
    payload[7] = (uint8_t)(hum_enc >> 24);
    can_send(can_node, can_id, payload, 8);
}

void can_send_gps(const CAN_NODE_t *can_node, uint16_t can_id,
                  float lat, float lon) {
    int32_t lat_enc = (int32_t)(lat * 1e6f);
    int32_t lon_enc = (int32_t)(lon * 1e6f);
    uint8_t payload[8];
    payload[0] = (uint8_t)(lat_enc);
    payload[1] = (uint8_t)(lat_enc >> 8);
    payload[2] = (uint8_t)(lat_enc >> 16);
    payload[3] = (uint8_t)(lat_enc >> 24);
    payload[4] = (uint8_t)(lon_enc);
    payload[5] = (uint8_t)(lon_enc >> 8);
    payload[6] = (uint8_t)(lon_enc >> 16);
    payload[7] = (uint8_t)(lon_enc >> 24);
    can_send(can_node, can_id, payload, 8);
}
```

4.5. EEPROM Settings - `eeprom_header_t` Union

A union maps the typed settings struct directly onto a byte array, so `eeprom_write` and `eeprom_read` can be called with `settings.as_bytes` without any casts:

```c
typedef union {
    struct __attribute__((packed)) eeprom_header_fields {
        uint8_t  magic;
        uint32_t timer_interval;
        uint16_t can_filter_id;
        uint8_t  can_filter_active;
        uint8_t  lcd_mode;
    } as_fields;
    uint8_t as_bytes[sizeof(struct eeprom_header_fields)];
} eeprom_header_t;
```

`cli_save_settings` populates `settings.as_fields.*` and writes `settings.as_bytes` to the EEPROM in a single call, followed by the two LCD text row buffers. `cli_load_settings` reads the raw bytes, checks `as_fields.magic`, and silently returns without applying anything if validation fails.

5. Commands

The system communicates over UART at **9600 8N1**. Open `apps/docklight.ptp` for pre-configured send sequences. On startup, the system prints the group header:

```
1  Grupo 3, Diogo Nogueira/Vasco Magolo, 1241692/1231562
```

Command	Action
↵	Carriage return - re-displays the menu
`0`	Exit (halt)
`1`	Read potentiometer ADC value (0 - 255)
`2` + value + ↵	Set LED blink full-period in ms (100 - 10 000)
`2a` → `2000`↵	Preset: 2 s period
`2b` → `1000`↵	Preset: 1 s period
`2c` → `500`↵	Preset: 500 ms period
`3`	Read current blink period
`4`	Trigger AHT10 read - print temperature and humidity
`5` + ID + ↵	Set CAN RX software filter (hex CAN ID, e.g. `4c0`)
`5a` → `4c0`↵	Preset: filter Group 3 frames (`0x4C0`)
`5b` → `4c3`↵	Preset: filter another group
`5c` → `7ff`↵	Preset: custom CAN filter
`6`	Enter LCD mode sub-menu
`6a` → `0`	LCD: off
`6b` → `1`	LCD: CAN RX sensor mode (last received filtered frame)
`6c` → `2`	LCD: potentiometer mode
`6d` → `3`	LCD: GPS mode (lat/lon or no-fix with satellite count)
`6e` → `4`	LCD: local AHT10 mode (direct sensor read)
`6f` → `5`	LCD: EEPROM text mode (displays rows set by `7`/`8`)
`7` + text + ↵	Write LCD row 1 text (max 16 chars, saved to EEPROM)
`8` + text + ↵	Write LCD row 2 text (max 16 chars, saved to EEPROM)

Table 7: Available commands via UART / Docklight

Example output for a GPS fix:

```
1  [GPS] 14:32:07Z  Lat: 41.198036  Lon: -8.652410  Satellites: 7
```

No-fix output:

```
1  [GPS] 00:00:00Z  No fix  Satellites: 3
```

Decoded CAN sensor frame (after filter is set):

```
1  [CAN 0x4C0] Temp: 23.4 C  Hum: 61.2 %
```

6. Electrical Schematic

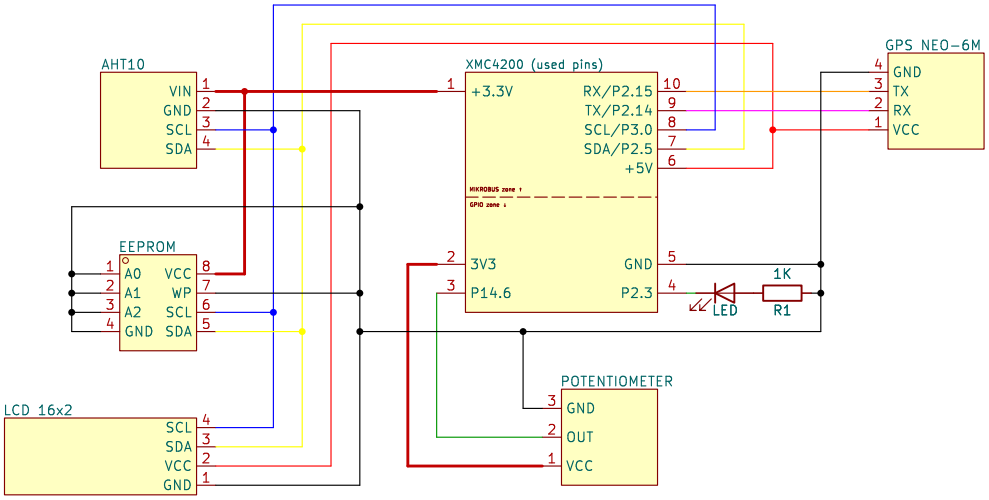

Figure 3: Electrical schematic

Wiring notes:

- **I2C bus:** SCL/P3.0 and SDA/P2.5 are shared between the AHT10 (0x70), the LCD (0x27), and the EEPROM (0xA0); all three connect in parallel on the same two wires.
- **Power:** AHT10 and EEPROM are powered from 3.3 V; LCD and GPS from 5 V. Power rails are shared buses - multiple devices connect to the same rail in parallel.